CAP 4105 — Machine Learning

Bonus Homework: Neural Networks

Due: April 17, 2026, 11:59 PM via Canvas
Points: 5 bonus points (added directly to your final course grade)
Individual work only — this is a bonus assignment; no group submissions
Submission: One `.ipynb` notebook file via Canvas

Overview

In this homework, you will implement a simple neural network **from scratch** using only Python and NumPy — no sklearn, no PyTorch, no TensorFlow. You will build a 2-layer MLP step by step, print the intermediate values at each stage, and observe how the network learns through training. A starter notebook `bonus_nn_starter.ipynb` is provided on Canvas.

AI Policy: You may use AI tools to help debug your code. However, make sure you **understand** every line you write — the print outputs will show whether the computation is correct, and similar questions may appear on Test 3.

Dataset

Use the 10-student dataset from class:

```
import numpy as np

X = np.array([
    [1, 5], [2, 6], [3, 4], [4, 7], [5, 8],
    [6, 7], [7, 8], [8, 3], [9, 6], [10, 4]
], dtype=float)

y = np.array([0, 0, 0, 1, 1, 1, 1, 1, 1, 1], dtype=float)

student_names = ['A','B','C','D','E','F','G','H','I','J']
```

Task 1: Sigmoid Function (1 point)

Implement the sigmoid function and its derivative.

```python
def sigmoid(z):
    # TODO: return 1 / (1 + exp(-z))
    pass

def sigmoid_derivative(z):
    # TODO: return sigmoid(z) * (1 - sigmoid(z))
    pass
```

Print checkpoint:

```python
for z in [-2, -1, 0, 1, 2, 3, 5]:
    print(f"σ({z:2d}) = {sigmoid(z):.4f},  σ'({z:2d}) = {sigmoid_derivative(z):.4f}")
```

Expected output (verify your implementation matches):

```
σ(-2) = 0.1192,  σ'(-2) = 0.1050
σ(-1) = 0.2689,  σ'(-1) = 0.1966
σ( 0) = 0.5000,  σ'( 0) = 0.2500
σ( 1) = 0.7311,  σ'( 1) = 0.1966
σ( 2) = 0.8808,  σ'( 2) = 0.1050
σ( 3) = 0.9526,  σ'( 3) = 0.0452
σ( 5) = 0.9933,  σ'( 5) = 0.0066
```

Task 2: Forward Pass (2 points)

Implement the forward pass for a 2-input, 2-hidden, 1-output MLP.

```python
def forward(x, weights):
    """
    x: array of shape (2,) — one data point [x1, x2]
    weights: dictionary with keys:
        'w11','w12','b1', 'w21','w22','b2', 'v1','v2','bo'
    Returns: dictionary with all intermediate values
        'z1','h1', 'z2','h2', 'zo','y_hat'
    """
    # Hidden neuron 1
    z1 = # TODO
    h1 = # TODO

    # Hidden neuron 2
    z2 = # TODO
    h2 = # TODO

```

```
    # Output neuron
    zo = # TODO
    y_hat = # TODO

    return {'z1': z1, 'h1': h1, 'z2': z2, 'h2': h2, 'zo': zo, 'y_hat':
y_hat}
```

Print checkpoint — use the weights from lecture slides:

```
weights = {
    'w11': 1, 'w12': 0, 'b1': -3,
    'w21': 0, 'w22': 1, 'b2': -6,
    'v1': 1, 'v2': 1, 'bo': -1
}

print("=== Forward Pass (Lecture Weights) ===")
print(f"{'Student':>8} {'x1':>4} {'x2':>4} | {'z1':>5} {'h1':>7} {'z2':>5}
{'h2':>7} | {'y_hat':>7} {'Pred':>5} {'True':>5}")
print("-" * 75)
for i in range(len(X)):
    r = forward(X[i], weights)
    pred = 1 if r['y_hat'] >= 0.5 else 0
    mark = 'O' if pred == y[i] else 'X'
    print(f"{student_names[i]:>8} {X[i,0]:4.0f} {X[i,1]:4.0f} | "
          f"{r['z1']:5.1f} {r['h1']:7.4f} {r['z2']:5.1f} {r['h2']:7.4f} |
"
          f"{r['y_hat']:7.4f} {pred:5d} {int(y[i]):5d} {mark}")
```

Expected: All 10 students should be classified correctly (10/10 accuracy).

Task 3: Loss Function (1 point)

Implement the binary cross-entropy loss.

```
def bce_loss(y_true, y_hat):
    """Binary cross-entropy for a single sample."""
    eps = 1e-8  # avoid log(0)
    # TODO: return -[y * log(y_hat + eps) + (1-y) * log(1 - y_hat + eps)]
    pass
```

Print checkpoint:

```
print("=== Loss per Student ===")
total_loss = 0
for i in range(len(X)):
    r = forward(X[i], weights)
    loss = bce_loss(y[i], r['y_hat'])
    total_loss += loss
    print(f"Student {student_names[i]}: y_hat = {r['y_hat']:.4f}, y =
{int(y[i])}, loss = {loss:.4f}")
print(f"\nAverage loss: {total_loss / len(X):.4f}")
```

Task 4: Backward Pass (3 points)

Implement the backward pass to compute gradients for all 9 parameters, for **one** data point.

```
def backward(x, y_true, fwd, weights):
    """
    x: input array (2,)
    y_true: true label
    fwd: dictionary from forward()
    weights: current weights
    Returns: dictionary of gradients for all 9 parameters
    """
    # Step 1: output error
    dL_dzo = # TODO: y_hat - y

    # Step 2: output layer gradients
    dL_dv1 = # TODO
    dL_dv2 = # TODO
    dL_dbo = # TODO

    # Step 3: propagate to hidden layer
    dL_dh1 = # TODO
    dL_dh2 = # TODO

    # Step 4: through sigmoid
    dL_dz1 = # TODO: dL_dh1 * sigmoid_derivative(fwd['z1'])
    dL_dz2 = # TODO

    # Step 5: hidden layer weight gradients
    dL_dw11 = # TODO
    dL_dw12 = # TODO
    dL_db1  = # TODO
    dL_dw21 = # TODO
    dL_dw22 = # TODO
    dL_db2  = # TODO
```

```
    return {
        'dv1': dL_dv1, 'dv2': dL_dv2, 'dbo': dL_dbo,
        'dw11': dL_dw11, 'dw12': dL_dw12, 'db1': dL_db1,
        'dw21': dL_dw21, 'dw22': dL_dw22, 'db2': dL_db2
    }
```

Print checkpoint — use Student F ($x_1 = 6, x_2 = 7, y = 1$):

```
fwd_F = forward(X[5], weights)   # Student F
grad_F = backward(X[5], y[5], fwd_F, weights)

print("=== Backward Pass: Student F (6, 7), y=1 ===")
print(f"y_hat = {fwd_F['y_hat']:.4f}, loss = {bce_loss(y[5],
fwd_F['y_hat']):.4f}")
print()
print(f"Output error (dL/dzo):  {fwd_F['y_hat'] - y[5]:.4f}")
print(f"sigma'(z1) = sigma'({fwd_F['z1']:.0f}) =
{sigmoid_derivative(fwd_F['z1']):.4f}")
print(f"sigma'(z2) = sigma'({fwd_F['z2']:.0f}) =
{sigmoid_derivative(fwd_F['z2']):.4f}")
print()
print("Gradients:")
for name, val in grad_F.items():
    print(f"  {name:5s} = {val: .4f}")
```

Expected output for verification (from lecture):

```
dv1   = -0.3196
dv2   = -0.2452
dbo   = -0.3355
dw11  = -0.0909
dw12  = -0.1061
db1   = -0.0152
dw21  = -0.3957
dw22  = -0.4617
db2   = -0.0660
```

Task 5: Training Loop (2 points)

Put it all together: train the network on all 10 students using gradient descent.

```
def train(X, y, lr=0.1, epochs=500):
    """
    Train the 2-2-1 MLP from random initialization.
    Returns: (final_weights, loss_history)
```

```
    Returns: (final_weights, loss_history)
    """
    # Random initialization
    np.random.seed(42)
    w = {
        'w11': np.random.randn()*0.5, 'w12': np.random.randn()*0.5, 'b1': 0.0,
        'w21': np.random.randn()*0.5, 'w22': np.random.randn()*0.5, 'b2': 0.0,
        'v1':  np.random.randn()*0.5, 'v2':  np.random.randn()*0.5, 'bo': 0.0
    }

    loss_history = []

    for epoch in range(epochs):
        total_loss = 0
        # Accumulate gradients over all samples
        grad_sum = {k: 0.0 for k in ['dv1','dv2','dbo','dw11','dw12','db1','dw21','dw22','db2']}

        for i in range(len(X)):
            fwd = forward(X[i], w)
            total_loss += bce_loss(y[i], fwd['y_hat'])
            grad = backward(X[i], y[i], fwd, w)
            for k in grad_sum:
                grad_sum[k] += grad[k]

        # Average gradients
        for k in grad_sum:
            grad_sum[k] /= len(X)

        # TODO: Update all 9 weights
        # w['w11'] -= lr * grad_sum['dw11']
        # ... (complete for all 9 parameters)

        avg_loss = total_loss / len(X)
        loss_history.append(avg_loss)

        # Print every 100 epochs
        if epoch % 100 == 0 or epoch == epochs - 1:
            correct = sum(1 for i in range(len(X))
                          if (forward(X[i], w)['y_hat'] >= 0.5) == y[i])
            print(f"Epoch {epoch:4d} | Loss: {avg_loss:.4f} | Accuracy: {correct}/{len(X)}")

    return w, loss_history
```

Run training and print final results:

Run training and print final results:

```
final_weights, losses = train(X, y, lr=0.5, epochs=1000)

print("\n=== Final Trained Weights ===")
for k, v in final_weights.items():
    print(f"  {k:4s} = {v: .4f}")

print("\n=== Final Predictions ===")
for i in range(len(X)):
    r = forward(X[i], final_weights)
    pred = 1 if r['y_hat'] >= 0.5 else 0
    mark = 'O' if pred == y[i] else 'X'
    print(f"Student {student_names[i]}: y_hat = {r['y_hat']:.4f} -> {pred}
(true: {int(y[i])}) {mark}")
```

Plot the loss curve:

```
import matplotlib.pyplot as plt
plt.plot(losses)
plt.xlabel('Epoch')
plt.ylabel('Average Loss')
plt.title('Training Loss Curve')
plt.grid(True, alpha=0.3)
plt.show()
```

Task 6: Observation Question (1 point)

After completing all tasks above, answer the following in a **Markdown cell** in your notebook (2–3 sentences each):

(a) Look at the printed $\sigma'(z_1)$ and $\sigma'(z_2)$ values from Task 4. Which neuron has the smaller gradient? What problem does this illustrate, and what activation function would fix it?

(b) Look at the loss curve from Task 5. Does the loss decrease smoothly or fluctuate? Did the network reach 100% accuracy on the 10 students? If not, why might a 2-hidden-neuron network struggle?

Grading

Task	Points	What We Check
Task 1: Sigmoid + derivative	1	Print output matches expected values
Task 2: Forward pass	2	Print table shows correct intermediate values; 10/10 accuracy with lecture weights
Task 3: Loss function	1	Loss values are reasonable (positive, larger for wrong predictions)
Task 4: Backward pass	3	Gradient values match expected output for Student F
Task 5: Training loop	2	Loss curve decreases; final accuracy printed
Task 6: Observation	1	Answers demonstrate understanding of vanishing gradient and training behavior
Total	**10**	

Tips

- **Start from Task 1 and go in order.** Each task builds on the previous one — if your sigmoid is wrong, everything downstream will be wrong.
- **Use the print checkpoints.** They exist to help you catch bugs early. If your numbers don't match the expected output, fix it before moving on.
- **The expected gradient values in Task 4 are your best friends.** If your `dv1` matches -0.3196, your output layer backprop is correct. If not, re-check the chain rule step by step.
- **For Task 5**, a learning rate of 0.5 and 1000 epochs should work. If the loss doesn't decrease, double-check your weight update signs (it should be `w -= lr * gradient`, not `+=`).
- The network has only 2 hidden neurons — it may not reach 100% accuracy on all 10 students, and that's okay. The point is to observe the training process.